

42 BEGINNER RESOURCES

A practical learning guide for your first steps at 42

Terminal • Typing • C • Git & GitHub

How to use this guide

Use this document as a companion to the introductory 42 videos. You do not need to learn everything before starting your first project. Explore one topic at a time, practise what you learn, and return to these resources whenever you get stuck.

Contents

- 1. Terminal & Shell
- 2. Typing Practice
- 3. C Programming
- 4. Git & GitHub
- 5. Recommended Learning Path
- 6. Final Advice



Tip: *A note for beginners — feeling confused at the beginning is completely normal. 42 is built around self-directed learning and peer-to-peer collaboration. Your goal is not to know everything immediately — your goal is to become comfortable finding answers, testing ideas, and learning from mistakes.*

1. Terminal & Shell

Terminal is one of the most important tools you'll use every day at 42. At first, the command line might feel unfamiliar, but with daily use, the basic commands will quickly become natural.

Essential commands

Command	What it does
<code>pwd</code>	Shows the current directory path you are in.
<code>ls</code>	Displays the contents of the current directory.
<code>cd</code>	Navigates from one directory to another.
<code>cd ..</code>	Goes up one directory level.
<code>mkdir</code>	Creates a new directory.
<code>touch</code>	Creates a new empty file.
<code>cat</code>	Displays the contents of a file.
<code>cp</code>	Copies a file or directory.
<code>mv</code>	Moves or renames a file or directory.
<code>rm</code>	Deletes a file. Use with caution.
<code>clear</code>	Clears the terminal screen.
<code>history</code>	Shows previously used commands.
<code>man</code>	Opens a command's manual to explore its features.

Learn & practise

Linux Journey — Interactive learning for Linux and command line.

The Missing Semester — Shell — In-depth explanation of the shell and command line.

OverTheWire: Bandit — A game environment where you learn Linux commands by solving practical challenges.

 **Tip:** Create a demo directory, create several files in it, change directories, rename and move files. Try to do everything using only the terminal.

2. Typing Practice

You'll spend a lot of time at 42 in the terminal and code editor. Fast and accurate typing can significantly make your work easier and let you focus more on solving problems.

Recommended tools

Monkeytype — Develop typing speed and accuracy.

TypingClub — Structured typing lessons for beginners.

Keybr — Helps develop touch typing and learn without looking at the keyboard.

10FastFingers — Typing speed tests and speed measurement.

 **Tip:** *At first, don't focus only on speed. It's more important to learn to type correctly with as few errors as possible.*

3. C Programming

C is the foundation of many core projects at 42. If you want to work more confidently on your first projects, it's useful to strengthen your understanding of C fundamentals and learn to explore documentation on your own.

Core topics to understand

- Variables & Data Types
- Conditions
- Loops
- Functions
- Arrays
- Strings
- Pointers
- Memory Allocation
- Structures
- Header Files

Learn C

Learn-C.org — Interactive C tutorial with the ability to test code immediately.

W3Schools — C — Simple and accessible explanations of C's fundamental concepts.

Beej's Guide to C Programming — More in-depth material on C programming.

cppreference — C Language — C language reference for documentation and quick reference.

Video learning

freeCodeCamp.org — **YouTube** — Long courses and tutorials on programming and computer science topics.

Core Dumped — **YouTube** — Interesting content on C programming and computer science topics.

 **Tip:** Don't limit yourself to just watching videos. After each new topic, write a small program yourself, modify it and try to predict what the output will be.

4. Git & GitHub

Git is a version control system that allows you to track changes in code, maintain different versions, and collaborate on projects. GitHub is a widely used online platform for Git repositories.

Essential Git commands

Command	What it does
<code>git init</code>	Creates a new Git repository.
<code>git status</code>	Shows the current state of the repository.
<code>git add</code>	Adds changes to the staging area.
<code>git commit</code>	Saves staged changes as a commit.
<code>git log</code>	Shows the commit history.
<code>git branch</code>	Creates and manages branches.
<code>git checkout</code>	Switches branches or performs other checkout operations.
<code>git switch</code>	Switches branches with modern and simple syntax.
<code>git clone</code>	Clones a remote repository.
<code>git pull</code>	Fetches and integrates remote changes.
<code>git push</code>	Sends local commits to a remote repository.
<code>git remote</code>	Manages remote repository addresses.

Learn Git

Learn Git Branching — A very interesting interactive game for learning Git and branching.

GitHub Skills — GitHub's official interactive courses and hands-on exercises.

Git Documentation — Git's official documentation.

GitHub Docs — GitHub's official guides and documentation.

 **Tip:** Create a small repository, add a C program, make at least 3 commits, create a branch and try to merge it.

5. Recommended Learning Path

If you are new to 42, you can follow this simple sequence. Don't try to learn everything in one day. The key is that at each step, you actually practice what you learn.

- 01 Learn basic Terminal commands** — Learn navigation, files and basic shell commands.
- 02 Practise typing** — Spend a few minutes each day working on typing speed and accuracy.
- 03 Learn C fundamentals** — Strengthen your understanding of functions, arrays, strings, pointers and memory fundamentals.
- 04 Learn Git basics** — Learn the concepts of repository, commit, branch, pull and push.
- 05 Start your first 42 project** — Start working, explore the Subject and learn along the way.

6. Final Advice

To succeed at 42, you don't need to know everything before starting your first project. Start working, explore what you don't know, try different solutions, ask your peers for help and learn from your mistakes.

 **Tip:** *Remember — you don't need to know everything before starting. Start working, search for what you don't know, ask your peers for help, and learn along the way.*

A simple weekly habit

- Spend a few minutes practising Terminal commands.
- Practise typing regularly, focusing on accuracy first.
- Write small C programs instead of only watching tutorials.
- Use Git for your own practice projects and learn from your commit history.
- When you get stuck, first try to understand the problem yourself, then use documentation and ask a peer for help.

Good luck with your first projects at 42!

Learn • Practise • Collaborate • Keep going